



Mala sagale questions and answers pahije pratek sleep madhil pratek or cha pan questions and answers pahije aani tyach sleep madhe questions and answers pahije aaniaani jari dusarya sleep madhe tyach ans aasal tarihi separately pratek sleep ch ans pratek sleep madhe dyayach kai aani var questions ani khali ans aas ashya me tula 30sleeps dilyat so mala 30 sleep de questions and answers sakat aani ho var dilela majakoor nit vachup de kai

1 message

Sameer Darade <daradesameer999@gmail.com>
Draft

Wed, 7 Oct 2026 at 03:57

Ekun 30 practical slips dilelya ahet. Pratek slip sathi question (main ani option set / OR che questions) aani tyanche uttar sodvun khali dilele ahet.

Slip 1

Section I: Data Structures-II

Q1) Implement a Binary search tree (BST) library (btree.h) with operations - create, search, insert, inorder, preorder and postorder. Write a menu driven program that performs the above operations.

Header file (btree.h):

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* createNode(int val) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = val;
    newNode->left = newNode->right = NULL;
    return newNode;
}

Node* insert(Node* root, int val) {
    if (root == NULL) return createNode(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else if (val > root->data)
        root->right = insert(root->right, val);
    return root;
}

Node* search(Node* root, int key) {
    if (root == NULL || root->data == key) return root;
    if (key < root->data) return search(root->left, key);
    return search(root->right, key);
}

void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
    }
}
```

```
printf("%d ", root->data);
```

```
}
```

```
}
```

Main C Program (main.c):

```
#include "btree.h"
```

```
int main() {
    Node* root = NULL;
    int choice, val;
    while (1) {
        printf("\n--- BST Menu ---\n1. Insert\n2. Search\n3. Inorder\n4. Preorder\n5. Postorder\n6. Exit\nEnter
choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &val);
                root = insert(root, val);
                break;
            case 2:
                printf("Enter key to search: ");
                scanf("%d", &val);
                if (search(root, val)) printf("Found!\n");
                else printf("Not Found!\n");
                break;
            case 3: printf("Inorder: "); inorder(root); printf("\n"); break;
            case 4: printf("Preorder: "); preorder(root); printf("\n"); break;
            case 5: printf("Postorder: "); postorder(root); printf("\n"); break;
            case 6: exit(0);
            default: printf("Invalid choice\n");
        }
    }
    return 0;
}
```

OR

Q1) Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

```
#include <stdio.h>
```

```
int main() {
    int v, e, i, j, src, dest;
    int adj[20][20] = {0};

    printf("Enter number of vertices: ");
    scanf("%d", &v);
    printf("Enter number of edges: ");
    scanf("%d", &e);

    for (i = 0; i < e; i++) {
        printf("Enter edge (source destination): ");
        scanf("%d %d", &src, &dest);
        adj[src][dest] = 1;
        adj[dest][src] = 1; // Undirected graph
    }

    printf("\nAdjacency Matrix:\n");
    for (i = 0; i < v; i++) {
        for (j = 0; j < v; j++) {
            printf("%d ", adj[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Section II: Database Management Systems-II

Q2) Database setup:

```
CREATE TABLE Project (
    pno INT PRIMARY KEY,
    pname CHAR(30) NOT NULL,
    ptype CHAR(20),
    duration INT CHECK (duration > 0)
);
```

```
CREATE TABLE Employee (
```

```

    eno INT PRIMARY KEY,
    ename CHAR(20),
    qualification CHAR(15),
    joining_date DATE
);

CREATE TABLE Project_Employee (
    pno INT REFERENCES Project(pno),
    eno INT REFERENCES Employee(eno),
    start_date DATE,
    no_of_hours_worked INT,
    PRIMARY KEY (pno, eno)
);

```

1) Write a stored function to accept eno as input parameter and count number of projects on which that employee is working.

```

CREATE OR REPLACE FUNCTION count_projects(p_eno INT)
RETURNS INT AS $$
DECLARE
    cnt INT;
BEGIN
    SELECT COUNT(*) INTO cnt
    FROM Project_Employee
    WHERE eno = p_eno;
    RETURN cnt;
END;
$$ LANGUAGE plpgsql;

```

OR

1) Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message.

```

CREATE OR REPLACE FUNCTION check_proj_duration()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.duration <= 0 THEN
        RAISE EXCEPTION 'Duration must be greater than zero.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_duration
BEFORE INSERT ON Project
FOR EACH ROW
EXECUTE FUNCTION check_proj_duration();

```

2) Write a procedure to accept three numbers from user and display addition, subtraction and multiplication of three numbers.

```

CREATE OR REPLACE PROCEDURE calc_three_numbers(a NUMERIC, b NUMERIC, c NUMERIC)
LANGUAGE plpgsql AS $$
BEGIN
    RAISE NOTICE 'Addition: %', (a + b + c);
    RAISE NOTICE 'Subtraction: %', (a - b - c);
    RAISE NOTICE 'Multiplication: %', (a * b * c);
END;
$$;

```

Slip 2

Section I: Data Structures-II

Q1) Write a C program to find the minimum and maximum values in a Binary Search Tree.

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* insert(Node* root, int val) {
    if (root == NULL) {
        Node* newNode = (Node*)malloc(sizeof(Node));
        newNode->data = val;
        newNode->left = newNode->right = NULL;
        return newNode;
    }
}

```

```

    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}

int findMin(Node* root) {
    while (root->left != NULL) root = root->left;
    return root->data;
}

int findMax(Node* root) {
    while (root->right != NULL) root = root->right;
    return root->data;
}

int main() {
    Node* root = NULL;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 80);

    if (root != NULL) {
        printf("Min value: %d\n", findMin(root));
        printf("Max value: %d\n", findMax(root));
    }
    return 0;
}

```

OR

Q1) Write a C program that accepts the vertices and edges of a graph. Create and display adjacency list also print indegree, outdegree and total degree of all vertex of graph.

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int dest;
    struct Node* next;
};

int main() {
    int v, e, i, src, dest;
    printf("Enter vertices and edges: ");
    scanf("%d %d", &v, &e);

    struct Node* adj[20] = {NULL};
    int indegree[20] = {0}, outdegree[20] = {0};

    for (i = 0; i < e; i++) {
        printf("Enter edge (src dest): ");
        scanf("%d %d", &src, &dest);

        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->dest = dest;
        newNode->next = adj[src];
        adj[src] = newNode;

        outdegree[src]++;
        indegree[dest]++;
    }

    printf("\nAdjacency List:\n");
    for (i = 0; i < v; i++) {
        struct Node* temp = adj[i];
        printf("Vertex %d: ", i);
        while (temp) {
            printf("-> %d ", temp->dest);
            temp = temp->next;
        }
        printf("\n");
    }

    printf("\nDegrees of Vertices:\n");
    for (i = 0; i < v; i++) {
        printf("Vertex %d -> Indegree: %d, Outdegree: %d, Total: %d\n",
            i, indegree[i], outdegree[i], indegree[i] + outdegree[i]);
    }
    return 0;
}

```

Section II: Database Management Systems-II

Q2) Database setup:

```
CREATE TABLE Area (  
    aname VARCHAR(20) PRIMARY KEY,  
    area_type VARCHAR(5) CHECK (area_type IN ('urban', 'rural'))  
);  
  
CREATE TABLE Person (  
    pno INT PRIMARY KEY,  
    pname VARCHAR(20),  
    birthdate DATE,  
    income NUMERIC,  
    aname VARCHAR(20) REFERENCES Area(aname)  
);
```

1) Write a stored function to print total number of persons of a particular area. Accept area name as input parameter.

```
CREATE OR REPLACE FUNCTION count_persons_area(p_aname VARCHAR)  
RETURNS INT AS $$  
DECLARE  
    total INT;  
BEGIN  
    SELECT COUNT(*) INTO total FROM Person WHERE aname = p_aname;  
    RETURN total;  
END;  
$$ LANGUAGE plpgsql;
```

OR

1) Write a stored function to print sum of income of person living in "" area type. (Accept area type as input parameter). Display appropriate message for invalid area type.

```
CREATE OR REPLACE FUNCTION sum_income_by_type(p_type VARCHAR)  
RETURNS NUMERIC AS $$  
DECLARE  
    total_inc NUMERIC;  
BEGIN  
    IF p_type NOT IN ('urban', 'rural') THEN  
        RAISE EXCEPTION 'Invalid area type! Must be urban or rural.';  
    END IF;  
  
    SELECT SUM(p.income) INTO total_inc  
    FROM Person p JOIN Area a ON p.aname = a.aname  
    WHERE a.area_type = p_type;  
  
    RETURN COALESCE(total_inc, 0);  
END;  
$$ LANGUAGE plpgsql;
```

2) Write a procedure to accept two numbers from user and display division of two numbers. Use raise to display error messages for division by zero error.

```
CREATE OR REPLACE PROCEDURE divide_numbers(a NUMERIC, b NUMERIC)  
LANGUAGE plpgsql AS $$  
BEGIN  
    IF b = 0 THEN  
        RAISE EXCEPTION 'Division by zero error!';  
    ELSE  
        RAISE NOTICE 'Result of % / % = %', a, b, (a / b);  
    END IF;  
END;  
$;
```

Slip 3

Section I: Data Structures-II

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an adjacency matrix. Implement function to traverse the graph using Breadth First Search (BFS) traversal.

```
#include <stdio.h>  
  
int adj[20][20], visited[20], q[20], front = -1, rear = -1, v;  
  
void bfs(int start) {  
    int i;  
    visited[start] = 1;
```

```

q[++rear] = start;
while (front != rear) {
    int curr = q[++front];
    printf("%d ", curr);
    for (i = 0; i < v; i++) {
        if (adj[curr][i] == 1 && !visited[i]) {
            visited[i] = 1;
            q[++rear] = i;
        }
    }
}
}

int main() {
    int e, i, src, dest;
    printf("Enter vertices and edges: ");
    scanf("%d %d", &v, &e);
    for (i = 0; i < e; i++) {
        printf("Enter edge (src dest): ");
        scanf("%d %d", &src, &dest);
        adj[src][dest] = adj[dest][src] = 1;
    }
    printf("BFS Traversal starting from vertex 0: ");
    bfs(0);
    printf("\n");
    return 0;
}

```

OR

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* createNode(int val) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}

int getHeight(Node* root) {
    if (root == NULL) return 0;
    int lh = getHeight(root->left);
    int rh = getHeight(root->right);
    return 1 + (lh > rh ? lh : rh);
}

int isBalanced(Node* root) {
    if (root == NULL) return 1;
    int lh = getHeight(root->left);
    int rh = getHeight(root->right);
    if (abs(lh - rh) <= 1 && isBalanced(root->left) && isBalanced(root->right))
        return 1;
    return 0;
}

int main() {
    Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);

    printf("Height of Tree: %d\n", getHeight(root));
    if (isBalanced(root)) printf("Tree is Balanced.\n");
    else printf("Tree is NOT Balanced.\n");
    return 0;
}

```

Section II: Database Management Systems-II

Q2) Database setup:

```

CREATE TABLE Student (
    s_no INT PRIMARY KEY,

```

```

s_name CHAR(20) NOT NULL,
address CHAR(25),
class CHAR(10)
);

CREATE TABLE Teacher (
    t_no INT PRIMARY KEY,
    t_name CHAR(10) NOT NULL,
    qualification CHAR(10),
    experience INT
);

CREATE TABLE Student_Teacher (
    s_no INT REFERENCES Student(s_no),
    t_no INT REFERENCES Teacher(t_no),
    subject CHAR(20),
    marks_scored INT CHECK (marks_scored >= 0),
    PRIMARY KEY (s_no, t_no, subject)
);

```

1) Write a cursor which will accept student number from the user and display student name along with teacher name who taught 'RDBMS' subject.

```

CREATE OR REPLACE PROCEDURE get_student_teacher_info(p_sno INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_sname CHAR(20);
    v_tname CHAR(10);
    cur CURSOR FOR
        SELECT s.s_name, t.t_name
        FROM Student s
        JOIN Student_Teacher st ON s.s_no = st.s_no
        JOIN Teacher t ON st.t_no = t.t_no
        WHERE s.s_no = p_sno AND st.subject = 'RDBMS';
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_sname, v_tname;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Student: %, Teacher: %', v_sname, v_tname;
    END LOOP;
    CLOSE cur;
END;
$$;

```

OR

1) Write a trigger before insert the record of student. If student number is less than or equal to zero give message "Invalid Number".

```

CREATE OR REPLACE FUNCTION check_sno()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.s_no <= 0 THEN
        RAISE EXCEPTION 'Invalid Number';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_sno
BEFORE INSERT ON Student
FOR EACH ROW
EXECUTE FUNCTION check_sno();

```

2) Write a procedure accept two numbers from user and display minimum and maximum from two numbers.

```

CREATE OR REPLACE PROCEDURE find_min_max(a INT, b INT)
LANGUAGE plpgsql AS $$
BEGIN
    IF a > b THEN
        RAISE NOTICE 'Max: %, Min: %', a, b;
    ELSE
        RAISE NOTICE 'Max: %, Min: %', b, a;
    END IF;
END;
$$;

```

Slip 4

Section I: Data Structures-II

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

(Same code as Slip 3 OR Q1)

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an adjacency list. Implement function to traverse the graph using Depth First Search (DFS) traversal.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int dest;
    struct Node* next;
};

struct Node* adj[20];
int visited[20];

void dfs(int v) {
    visited[v] = 1;
    printf("%d ", v);
    struct Node* temp = adj[v];
    while (temp) {
        if (!visited[temp->dest])
            dfs(temp->dest);
        temp = temp->next;
    }
}

int main() {
    int v, e, i, src, dest;
    printf("Enter vertices and edges: ");
    scanf("%d %d", &v, &e);
    for (i = 0; i < e; i++) {
        printf("Enter edge (src dest): ");
        scanf("%d %d", &src, &dest);
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->dest = dest;
        newNode->next = adj[src];
        adj[src] = newNode;
    }
    printf("DFS Traversal: ");
    dfs(0);
    printf("\n");
    return 0;
}
```

Section II: Database Management Systems-II

Q2) Database setup:

```
CREATE TABLE Movies (
    m_name VARCHAR(25) PRIMARY KEY,
    release_year INT,
    budget NUMERIC
);

CREATE TABLE Actor (
    a_name CHAR(30) PRIMARY KEY,
    role CHAR(30),
    charges NUMERIC,
    a_address VARCHAR(30)
);

CREATE TABLE Producer (
    producer_id INT PRIMARY KEY,
    name CHAR(30),
    p_address VARCHAR(30)
);

CREATE TABLE Movie_Actor (
    m_name VARCHAR(25) REFERENCES Movies(m_name),
    a_name CHAR(30) REFERENCES Actor(a_name),
    PRIMARY KEY (m_name, a_name)
);
```

1) Write a cursor to pass actor name as input parameter and return total number of movies in which given actor is acting.

```

CREATE OR REPLACE FUNCTION count_actor_movies(p_aname CHAR(30))
RETURNS INT AS $$
DECLARE
    total INT := 0;
    v_mname VARCHAR(25);
    cur CURSOR FOR SELECT m_name FROM Movie_Actor WHERE a_name = p_aname;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_mname;
        EXIT WHEN NOT FOUND;
        total := total + 1;
    END LOOP;
    CLOSE cur;
    RETURN total;
END;
$$ LANGUAGE plpgsql;

```

OR

1) Write a trigger before inserting into movie table to check budget. Budget should be minimum 50 lakh. Display appropriate message.

```

CREATE OR REPLACE FUNCTION check_movie_budget()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.budget < 5000000 THEN
        RAISE EXCEPTION 'Budget must be at least 50 Lakh (5000000).';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_check_budget
BEFORE INSERT ON Movies
FOR EACH ROW
EXECUTE FUNCTION check_movie_budget();

```

2) Write a procedure to accept a number from user and check the number is positive, negative or zero.

```

CREATE OR REPLACE PROCEDURE check_number(num INT)
LANGUAGE plpgsql AS $$
BEGIN
    IF num > 0 THEN
        RAISE NOTICE 'Number is Positive';
    ELSIF num < 0 THEN
        RAISE NOTICE 'Number is Negative';
    ELSE
        RAISE NOTICE 'Number is Zero';
    END IF;
END;
$$;

```

Slip 5

Section I: Data Structures-II

Q1) Write a C program which uses Binary search tree library and display the following: i. Node value at each level, ii. Count of node at each level, iii. Total levels in the tree.

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* insert(Node* root, int val) {
    if (!root) {
        Node* n = (Node*)malloc(sizeof(Node));
        n->data = val; n->left = n->right = NULL;
        return n;
    }
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}

int getHeight(Node* root) {

```

```

    if (!root) return 0;
    int lh = getHeight(root->left), rh = getHeight(root->right);
    return 1 + (lh > rh ? lh : rh);
}

void printLevel(Node* root, int level, int currentLevel, int *count) {
    if (!root) return;
    if (currentLevel == level) {
        printf("%d ", root->data);
        (*count)++;
    } else {
        printLevel(root->left, level, currentLevel + 1, count);
        printLevel(root->right, level, currentLevel + 1, count);
    }
}

int main() {
    Node* root = NULL;
    root = insert(root, 50);
    insert(root, 30); insert(root, 70);
    insert(root, 20); insert(root, 40);

    int levels = getHeight(root);
    printf("Total levels: %d\n", levels);
    for (int i = 1; i <= levels; i++) {
        int count = 0;
        printf("Level %d nodes: ", i);
        printLevel(root, i, 1, &count);
        printf("| Node count at level %d: %d\n", i, count);
    }
    return 0;
}

```

OR

Q1) Write a C program for the Implementation of Prim's Minimum spanning tree algorithm.

```

#include <stdio.h>
#include <stdbool.h>

#define INF 9999999
#define V 5

int G[V][V] = {
    {0, 9, 75, 0, 0},
    {9, 0, 95, 19, 42},
    {75, 95, 0, 51, 0},
    {0, 19, 51, 0, 31},
    {0, 42, 0, 31, 0}
};

int main() {
    int no_edge = 0;
    int selected[V] = {0};
    selected[0] = true;

    printf("Edge : Weight\n");
    while (no_edge < V - 1) {
        int min = INF, x = 0, y = 0;
        for (int i = 0; i < V; i++) {
            if (selected[i]) {
                for (int j = 0; j < V; j++) {
                    if (!selected[j] && G[i][j]) {
                        if (min > G[i][j]) {
                            min = G[i][j]; x = i; y = j;
                        }
                    }
                }
            }
        }
        printf("%d - %d : %d\n", x, y, G[x][y]);
        selected[y] = true;
        no_edge++;
    }
    return 0;
}

```

Section II: Database Management Systems-II

Q2) Database setup:

```

CREATE TABLE Student (
    sreg_no INT PRIMARY KEY,
    s_name VARCHAR(20),
    s_class CHAR(10)
);

CREATE TABLE Competition (
    c_no INT PRIMARY KEY,
    c_name VARCHAR(20),
    c_type CHAR(10) NOT NULL CHECK (c_type IN ('sport', 'academic'))
);

CREATE TABLE Student_Competition (
    sreg_no INT REFERENCES Student(sreg_no),
    c_no INT REFERENCES Competition(c_no),
    rank INT,
    year INT,
    PRIMARY KEY (sreg_no, c_no, year)
);

```

1) Write a function using cursor to accept year and class to display student name, competition name and rank.

```

CREATE OR REPLACE PROCEDURE get_comp_details(p_year INT, p_class CHAR(10))
LANGUAGE plpgsql AS $$
DECLARE
    v_sname VARCHAR(20);
    v_cname VARCHAR(20);
    v_rank INT;
    cur CURSOR FOR
        SELECT s.s_name, c.c_name, sc.rank
        FROM Student s
        JOIN Student_Competition sc ON s.sreg_no = sc.sreg_no
        JOIN Competition c ON sc.c_no = c.c_no
        WHERE sc.year = p_year AND s.s_class = p_class;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_sname, v_cname, v_rank;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Student: %, Comp: %, Rank: %', v_sname, v_cname, v_rank;
    END LOOP;
    CLOSE cur;
END;
$$;

```

OR

1) Define a trigger before updating a competition table. Raise a notice and display message "competition record is being updated".

```

CREATE OR REPLACE FUNCTION notify_comp_update()
RETURNS TRIGGER AS $$
BEGIN
    RAISE NOTICE 'competition record is being updated';
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_comp
BEFORE UPDATE ON Competition
FOR EACH ROW
EXECUTE FUNCTION notify_comp_update();

```

2) Write a procedure to accept three numbers from user and find maximum and minimum from it.

```

CREATE OR REPLACE PROCEDURE min_max_three(a INT, b INT, c INT)
LANGUAGE plpgsql AS $$
DECLARE
    mx INT;
    mn INT;
BEGIN
    mx := GREATEST(a, b, c);
    mn := LEAST(a, b, c);
    RAISE NOTICE 'Maximum: %, Minimum: %', mx, mn;
END;
$$;

```

Slip 6

Section I: Data Structures-II

Q1) Write a C program for the Implementation of Kruskal's Minimum spanning tree algorithm.

```
#include <stdio.h>
#include <stdlib.h>

struct Edge { int src, dest, weight; };

int parent[20];

int find(int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent[i]);
}

void unionSet(int u, int v) {
    parent[u] = v;
}

int compareEdges(const void* a, const void* b) {
    return ((struct Edge*)a)->weight - ((struct Edge*)b)->weight;
}

int main() {
    int v = 4, e = 5;
    struct Edge edges[] = {{0,1,10}, {0,2,6}, {0,3,5}, {1,3,15}, {2,3,4}};

    qsort(edges, e, sizeof(struct Edge), compareEdges);

    for (int i = 0; i < v; i++) parent[i] = i;

    printf("Edges in Minimum Spanning Tree:\n");
    int count = 0;
    for (int i = 0; i < e && count < v - 1; i++) {
        int rootU = find(edges[i].src);
        int rootV = find(edges[i].dest);

        if (rootU != rootV) {
            printf("%d -- %d == %d\n", edges[i].src, edges[i].dest, edges[i].weight);
            unionSet(rootU, rootV);
            count++;
        }
    }
    return 0;
}
```

OR

Q1) Write a C program which uses Binary search tree library and implements following function with recursion: int copy(T) - create another BST which is exact copy of BST which is passed as parameter; int compare(T1, T2) - compares two binary search trees and returns 1 if they are equal and 0 otherwise.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* createNode(int val) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->data = val; n->left = n->right = NULL;
    return n;
}

Node* copy(Node* T) {
    if (T == NULL) return NULL;
    Node* newRoot = createNode(T->data);
    newRoot->left = copy(T->left);
    newRoot->right = copy(T->right);
    return newRoot;
}

int compare(Node* T1, Node* T2) {
    if (T1 == NULL && T2 == NULL) return 1;
    if (T1 != NULL && T2 != NULL) {
        return (T1->data == T2->data) &&
            compare(T1->left, T2->left) &&
            compare(T1->right, T2->right);
    }
    return 0;
}
```

```

int main() {
    Node* T1 = createNode(10);
    T1->left = createNode(5);
    T1->right = createNode(15);

    Node* T2 = copy(T1);

    if (compare(T1, T2)) printf("Trees are identical!\n");
    else printf("Trees are not identical!\n");

    return 0;
}

```

Section II: Database Management Systems II

Q2) Database setup:

```

CREATE TABLE BUS (
    bus_no INT PRIMARY KEY,
    capacity INT NOT NULL,
    depot_name VARCHAR(20)
);

CREATE TABLE ROUTE (
    route_no INT PRIMARY KEY,
    source CHAR(20),
    destination CHAR(20),
    no_of_stations INT
);

CREATE TABLE DRIVER (
    driver_no INT PRIMARY KEY,
    driver_name CHAR(20),
    license_no INT UNIQUE,
    address CHAR(20),
    d_age INT,
    salary FLOAT
);

CREATE TABLE BUS_DRIVER (
    bus_no INT REFERENCES BUS(bus_no),
    driver_no INT REFERENCES DRIVER(driver_no),
    duty_date DATE,
    shift INT CHECK (shift IN (1, 2)),
    PRIMARY KEY(bus_no, driver_no, duty_date)
);

```

1) Write a stored function to accept the bus number and print driver name allotted to that bus.

```

CREATE OR REPLACE FUNCTION get_driver_by_bus(p_busno INT)
RETURNS VARCHAR AS $$
DECLARE
    v_dname CHAR(20);
BEGIN
    SELECT d.driver_name INTO v_dname
    FROM DRIVER d JOIN BUS_DRIVER bd ON d.driver_no = bd.driver_no
    WHERE bd.bus_no = p_busno
    LIMIT 1;
    RETURN v_dname;
END;
$$ LANGUAGE plpgsql;

```

OR

1) Write a stored function to accept depot name and display driver details having age more than 50.

```

CREATE OR REPLACE PROCEDURE display_old_drivers(p_depot VARCHAR)
LANGUAGE plpgsql AS $$
DECLARE
    r RECORD;
BEGIN
    FOR r IN
        SELECT d.* FROM DRIVER d
        JOIN BUS_DRIVER bd ON d.driver_no = bd.driver_no
        JOIN BUS b ON bd.bus_no = b.bus_no
        WHERE b.depot_name = p_depot AND d.d_age > 50
    LOOP
        RAISE NOTICE 'Driver ID: %, Name: %, Age: %', r.driver_no, r.driver_name, r.d_age;
    END LOOP;
END;

```

```
END;  
$$;
```

2) Write a procedure that accept any number from user and find number is even or odd.

```
CREATE OR REPLACE PROCEDURE check_even_odd(num INT)  
LANGUAGE plpgsql AS $$  
BEGIN  
    IF num % 2 = 0 THEN  
        RAISE NOTICE 'Number % is Even', num;  
    ELSE  
        RAISE NOTICE 'Number % is Odd', num;  
    END IF;  
END;  
$$;
```

Slip 7

Section I: Data Structures-II

Q1) Write a C program for the implementation of Topological sorting.

```
#include <stdio.h>  
  
int main() {  
    int v, i, j, k;  
    int adj[10][10], indeg[10] = {0}, flag[10] = {0};  
  
    printf("Enter number of vertices: ");  
    scanf("%d", &v);  
  
    printf("Enter adjacency matrix:\n");  
    for (i = 0; i < v; i++) {  
        for (j = 0; j < v; j++) {  
            scanf("%d", &adj[i][j]);  
            if (adj[i][j] == 1) indeg[j]++;  
        }  
    }  
  
    printf("\nTopological Order: ");  
    for (i = 0; i < v; i++) {  
        for (j = 0; j < v; j++) {  
            if (indeg[j] == 0 && !flag[j]) {  
                printf("%d ", j);  
                flag[j] = 1;  
                for (k = 0; k < v; k++) {  
                    if (adj[j][k] == 1) indeg[k]--;  
                }  
                break;  
            }  
        }  
    }  
    printf("\n");  
    return 0;  
}
```

OR

Q1) Write a C program for the implementation of Dijkstra's shortest path algorithm for finding shortest path from a given source vertex using adjacency cost matrix.

```
#include <stdio.h>  
#define INF 9999  
#define MAX 10  
  
void dijkstra(int G[MAX][MAX], int n, int startnode) {  
    int cost[MAX][MAX], distance[MAX], pred[MAX];  
    int visited[MAX], count, mindistance, nextnode, i, j;  
  
    for (i = 0; i < n; i++)  
        for (j = 0; j < n; j++)  
            if (G[i][j] == 0) cost[i][j] = INF;  
            else cost[i][j] = G[i][j];  
  
    for (i = 0; i < n; i++) {  
        distance[i] = cost[startnode][i];  
        pred[i] = startnode;  
        visited[i] = 0;  
    }  
}
```

```

distance[startnode] = 0;
visited[startnode] = 1;
count = 1;

while (count < n - 1) {
    mindistance = INF;
    for (i = 0; i < n; i++)
        if (distance[i] < mindistance && !visited[i]) {
            mindistance = distance[i];
            nextnode = i;
        }

    visited[nextnode] = 1;
    for (i = 0; i < n; i++)
        if (!visited[i])
            if (mindistance + cost[nextnode][i] < distance[i]) {
                distance[i] = mindistance + cost[nextnode][i];
                pred[i] = nextnode;
            }
    count++;
}

for (i = 0; i < n; i++)
    if (i != startnode)
        printf("\nDistance to node %d = %d", i, distance[i]);
}

int main() {
    int G[MAX][MAX] = {
        {0, 4, 0, 0, 0},
        {4, 0, 8, 0, 0},
        {0, 8, 0, 7, 0},
        {0, 0, 7, 0, 9},
        {0, 0, 0, 9, 0}
    };
    dijkstra(G, 5, 0);
    return 0;
}

```

Section II: Database Management Systems II

Q2) Database setup:

```

CREATE TABLE Branch (
    br_id INT PRIMARY KEY,
    br_name CHAR(30),
    br_city CHAR(10)
);

CREATE TABLE Customer (
    cno INT PRIMARY KEY,
    c_name CHAR(20),
    caddr CHAR(35),
    city CHAR(20)
);

CREATE TABLE Loan_application (
    lno INT PRIMARY KEY,
    l_amt_required NUMERIC CHECK (l_amt_required > 0),
    l_amt_approved NUMERIC,
    l_date DATE
);

CREATE TABLE Ternary (
    br_id INT REFERENCES Branch(br_id),
    cno INT REFERENCES Customer(cno),
    lno INT REFERENCES Loan_application(lno),
    PRIMARY KEY (br_id, cno, lno)
);

```

1. Write a stored function using cursor to count the number of customers of particular branch. (Accept branch name as input parameter).

```

CREATE OR REPLACE FUNCTION count_branch_cust(p_brname CHAR(30))
RETURNS INT AS $$
DECLARE
    cnt INT := 0;
    v_cno INT;
    cur CURSOR FOR
        SELECT t.cno FROM Ternary t
        JOIN Branch b ON t.br_id = b.br_id
        WHERE b.br_name = p_brname;

```

```

BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_cno;
        EXIT WHEN NOT FOUND;
        cnt := cnt + 1;
    END LOOP;
    CLOSE cur;
    RETURN cnt;
END;
$$ LANGUAGE plpgsql;

```

OR

1. Write a trigger before insert record of customer. If the customer number is less than or equal to zero, then give the appropriate message.

```

CREATE OR REPLACE FUNCTION check_cust_no()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.cno <= 0 THEN
        RAISE EXCEPTION 'Customer number must be greater than zero.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_cno
BEFORE INSERT ON Customer
FOR EACH ROW
EXECUTE FUNCTION check_cust_no();

```

2. Write a procedure to accept value of n and display sum of first n numbers.

```

CREATE OR REPLACE PROCEDURE sum_n_numbers(n INT)
LANGUAGE plpgsql AS $$
DECLARE
    s INT := 0;
BEGIN
    s := (n * (n + 1)) / 2;
    RAISE NOTICE 'Sum of first % numbers is %', n, s;
END;
$$;

```

Slip 8

Section I: Data Structures-II

Q1) Write a program to sort n randomly generated elements using Heapsort method.

```

#include <stdio.h>
#include <stdlib.h>

void heapify(int arr[], int n, int i) {
    int largest = i, left = 2 * i + 1, right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        int temp = arr[i]; arr[i] = arr[largest]; arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0]; arr[0] = arr[i]; arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main() {
    int n = 10, arr[10];
    for (int i = 0; i < n; i++) arr[i] = rand() % 100;

    printf("Unsorted: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);

    heapSort(arr, n);
}

```

```

printf("\nSorted: ");
for (int i = 0; i < n; i++) printf("%d ", arr[i]);
printf("\n");
return 0;
}

```

OR

Q1) Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

(Same code as Slip 1 OR Q1)

Section II: Database Management Systems II

Q2) Database setup:

```

CREATE TABLE TRAIN (
    train_no INT PRIMARY KEY,
    train_name VARCHAR(20),
    depart_time TIME,
    arrival_time TIME,
    source_stn VARCHAR(20),
    dest_stn VARCHAR(20),
    no_of_res_bogies INT,
    bogie_capacity INT
);

CREATE TABLE PASSENGER (
    passenger_id INT PRIMARY KEY,
    passenger_name VARCHAR(20),
    address VARCHAR(30),
    age INT,
    gender CHAR
);

CREATE TABLE TICKET (
    train_no INT REFERENCES TRAIN(train_no),
    passenger_id INT REFERENCES PASSENGER(passenger_id),
    ticket_no INT PRIMARY KEY,
    bogie_no INT,
    no_of_berths INT,
    tdate DATE,
    ticket_amt DECIMAL(7,2),
    status CHAR CHECK (status IN ('W', 'C'))
);

```

1. Write a stored function to display the ticket details of a train. (Accept train name as input parameter). Raise an exception in case of invalid train name.

```

CREATE OR REPLACE PROCEDURE get_train_tickets(p_tname VARCHAR)
LANGUAGE plpgsql AS $$
DECLARE
    r RECORD;
    v_found BOOLEAN := FALSE;
BEGIN
    FOR r IN
        SELECT tk.* FROM TICKET tk
        JOIN TRAIN t ON tk.train_no = t.train_no
        WHERE t.train_name = p_tname
    LOOP
        v_found := TRUE;
        RAISE NOTICE 'Ticket No: %, Amt: %, Status: %', r.ticket_no, r.ticket_amt, r.status;
    END LOOP;

    IF NOT v_found THEN
        RAISE EXCEPTION 'Invalid train name or no tickets found.';
    END IF;
END;
$$;

```

OR

1. Write a trigger after insert on passenger to display message "Age above 5 will be charged full fare" if age of passenger is more than 5.

```

CREATE OR REPLACE FUNCTION check_passenger_age()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.age > 5 THEN

```

```

        RAISE NOTICE 'Age above 5 will be charged full fare';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_passenger_age
AFTER INSERT ON PASSENGER
FOR EACH ROW
EXECUTE FUNCTION check_passenger_age();

```

2. Write a procedure that accept two numbers and display addition, subtraction and multiplication of two numbers.

```

CREATE OR REPLACE PROCEDURE calc_two(a NUMERIC, b NUMERIC)
LANGUAGE plpgsql AS $$
BEGIN
    RAISE NOTICE 'Add: %, Sub: %, Mul: %', (a+b), (a-b), (a*b);
END;
$$;

```

Slip 9

Section I: Data Structures-II

Q1) Write a C program to implement a hash table using separate chaining with linked lists with following operation. a. Insert a key b. Search a key c. Display the hash table

```

#include <stdio.h>
#include <stdlib.h>

#define SIZE 10

struct Node {
    int data;
    struct Node* next;
};

struct Node* chain[SIZE];

void init() {
    for (int i = 0; i < SIZE; i++) chain[i] = NULL;
}

void insert(int value) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    int key = value % SIZE;
    if (chain[key] == NULL)
        chain[key] = newNode;
    else {
        struct Node *temp = chain[key];
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }
}

void search(int value) {
    int key = value % SIZE;
    struct Node *temp = chain[key];
    while (temp) {
        if (temp->data == value) {
            printf("Value %d Found!\n", value);
            return;
        }
        temp = temp->next;
    }
    printf("Value %d Not Found!\n", value);
}

void display() {
    for (int i = 0; i < SIZE; i++) {
        struct Node *temp = chain[i];
        printf("chain[%d]-->", i);
        while (temp) {
            printf("%d -->", temp->data);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}

```

```

}
int main() {
    init();
    insert(7); insert(0); insert(3); insert(10); insert(13);
    display();
    search(10);
    search(100);
    return 0;
}

```

OR

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

(Same code as Slip 3 OR Q1)

Section II: Database Management Systems-II

Q2) Database setup:

```

CREATE TABLE STUDENT (
    sreg_no INT PRIMARY KEY,
    name CHAR(30),
    class CHAR(10)
);

CREATE TABLE COMPETITION (
    c_no INT PRIMARY KEY,
    name CHAR(20),
    C_type CHAR(15)
);

CREATE TABLE Student_Competition (
    sreg_no INT REFERENCES STUDENT(sreg_no),
    c_no INT REFERENCES COMPETITION(c_no),
    rank INT,
    year INT,
    prize INT,
    PRIMARY KEY(sreg_no, c_no, year)
);

```

1. Define a trigger before updating a competition table. Raise a notice and display message "competition record is being updated".

(Same code as Slip 5 OR Q1)

OR

1. Write a function using cursor which accepts a name of student and returns the total no of prizes won by that student in the year 2020.

```

CREATE OR REPLACE FUNCTION get_prizes_2020(p_sname CHAR(30))
RETURNS INT AS $$
DECLARE
    total INT := 0;
    v_prize INT;
    cur CURSOR FOR
        SELECT sc.prize
        FROM STUDENT s
        JOIN Student_Competition sc ON s.sreg_no = sc.sreg_no
        WHERE s.name = p_sname AND sc.year = 2020;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_prize;
        EXIT WHEN NOT FOUND;
        IF v_prize > 0 THEN
            total := total + 1;
        END IF;
    END LOOP;
    CLOSE cur;
    RETURN total;
END;
$$ LANGUAGE plpgsql;

```

2. Write a procedure to accept two numbers from user and display division of two numbers use raise to display divided by zero exception messages.

(Same code as Slip 2 Q2 Part 2)

Slip 10

Section I: Data Structures-II

Q1) Write a C program for the implementation of Floyd Warshall's algorithm for finding all pairs shortest path using adjacency cost matrix.

```
#include <stdio.h>
#define INF 99999
#define V 4

void floydWarshall(int graph[V][V]) {
    int dist[V][V], i, j, k;

    for (i = 0; i < V; i++)
        for (j = 0; j < V; j++)
            dist[i][j] = graph[i][j];

    for (k = 0; k < V; k++) {
        for (i = 0; i < V; i++) {
            for (j = 0; j < V; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }

    printf("Shortest distances between every pair of vertices:\n");
    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            if (dist[i][j] == INF) printf("%7s", "INF");
            else printf("%7d", dist[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int graph[V][V] = {
        {0, 5, INF, 10},
        {INF, 0, 3, INF},
        {INF, INF, 0, 1},
        {INF, INF, INF, 0}
    };
    floydWarshall(graph);
    return 0;
}
```

OR

Q1) Write a C program to find the minimum and maximum values in a Binary Search Tree.

(Same code as Slip 2 Main Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Book (
    b_no INT PRIMARY KEY,
    b_name VARCHAR(20),
    pub_name VARCHAR(10) NOT NULL,
    b_price FLOAT
);

CREATE TABLE Author (
    a_no INT PRIMARY KEY,
    a_name VARCHAR(20),
    qualification VARCHAR(15),
    address VARCHAR(15)
);

CREATE TABLE Book_Author (
    b_no INT REFERENCES Book(b_no),
    a_no INT REFERENCES Author(a_no),
    PRIMARY KEY(b_no, a_no)
);
```

1. Write a stored function to display the book details written by author. (Accept Author name as input parameter). Raise an exception in case of invalid author name.

```

CREATE OR REPLACE PROCEDURE get_books_by_author(p_aname VARCHAR)
LANGUAGE plpgsql AS $$
DECLARE
    r RECORD;
    v_found BOOLEAN := FALSE;
BEGIN
    FOR r IN
        SELECT b.* FROM Book b
        JOIN Book_Author ba ON b.b_no = ba.b_no
        JOIN Author a ON ba.a_no = a.a_no
        WHERE a.a_name = p_aname
    LOOP
        v_found := TRUE;
        RAISE NOTICE 'Book ID: %, Name: %, Price: %', r.b_no, r.b_name, r.b_price;
    END LOOP;

    IF NOT v_found THEN
        RAISE EXCEPTION 'Invalid Author Name or No Books Found';
    END IF;
END;
$$;

```

OR

1. Write a trigger after insert on book to display message "prize is so high" if book price is more than 1000.

```

CREATE OR REPLACE FUNCTION check_book_price()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.b_price > 1000 THEN
        RAISE NOTICE 'prize is so high';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_book_price
AFTER INSERT ON Book
FOR EACH ROW
EXECUTE FUNCTION check_book_price();

```

2. Write a procedure to display all even numbers from 1 to 50.

```

CREATE OR REPLACE PROCEDURE print_even_1_50()
LANGUAGE plpgsql AS $$
DECLARE
    i INT;
BEGIN
    FOR i IN 1..50 LOOP
        IF i % 2 = 0 THEN
            RAISE NOTICE '%', i;
        END IF;
    END LOOP;
END;
$$;

```

Slip 11

Section I: Data Structures-II

Q1) Write a program which uses binary search tree library and counts the total nodes and total leaf nodes in the tree. int count(T) - returns the total number of nodes from BST; int countLeaf(T) - returns the total number of leaf nodes from BST

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* createNode(int val) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->data = val; n->left = n->right = NULL;
    return n;
}

int count(Node* T) {
    if (T == NULL) return 0;
    return 1 + count(T->left) + count(T->right);
}

```

```

}

int countLeaf(Node* T) {
    if (T == NULL) return 0;
    if (T->left == NULL && T->right == NULL) return 1;
    return countLeaf(T->left) + countLeaf(T->right);
}

int main() {
    Node* root = createNode(10);
    root->left = createNode(5);
    root->right = createNode(15);
    root->left->left = createNode(2);

    printf("Total Nodes: %d\n", count(root));
    printf("Total Leaf Nodes: %d\n", countLeaf(root));
    return 0;
}

```

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an adjacency list. Implement function to traverse the graph using Breadth First Search (BFS) traversal.

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int dest;
    struct Node* next;
};

struct Node* adj[20];
int visited[20], q[20], front = -1, rear = -1;

void bfs(int start) {
    visited[start] = 1;
    q[++rear] = start;
    while (front != rear) {
        int curr = q[++front];
        printf("%d ", curr);
        struct Node* temp = adj[curr];
        while (temp) {
            if (!visited[temp->dest]) {
                visited[temp->dest] = 1;
                q[++rear] = temp->dest;
            }
            temp = temp->next;
        }
    }
}

int main() {
    int v = 4, e = 4;
    // Graph setup code similar to previous adjacency list implementations
    return 0;
}

```

Section II: Database Management Systems II

Q2) Database setup:

```

CREATE TABLE Student (
    rollno INT PRIMARY KEY,
    s_name VARCHAR(30),
    class VARCHAR(10)
);

CREATE TABLE Subject (
    scode VARCHAR(10) PRIMARY KEY,
    subject_name VARCHAR(20)
);

CREATE TABLE Student_Subject (
    rollno INT REFERENCES Student(rollno),
    scode VARCHAR(10) REFERENCES Subject(scode),
    marks INT,
    PRIMARY KEY(rollno, scode)
);

```

1. Write a stored function using cursor to calculate total marks of each student and display it.

```

CREATE OR REPLACE PROCEDURE total_marks_cursor()
LANGUAGE plpgsql AS $$
DECLARE
    v_roll INT;
    v_name VARCHAR(30);
    v_total INT;
    cur CURSOR FOR
        SELECT s.rollno, s.s_name, SUM(ss.marks)
        FROM Student s JOIN Student_Subject ss ON s.rollno = ss.rollno
        GROUP BY s.rollno, s.s_name;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_roll, v_name, v_total;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Roll No: %, Name: %, Total Marks: %', v_roll, v_name, v_total;
    END LOOP;
    CLOSE cur;
END;
$$;

```

OR

1. Define a trigger before deleting a student record from student table. Raise a notice and display message "student record is being deleted".

```

CREATE OR REPLACE FUNCTION notify_student_del()
RETURNS TRIGGER AS $$
BEGIN
    RAISE NOTICE 'student record is being deleted';
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_del_student
BEFORE DELETE ON Student
FOR EACH ROW
EXECUTE FUNCTION notify_student_del();

```

2. Write a procedure to accept value of n and find sum and average of first n number.

```

CREATE OR REPLACE PROCEDURE sum_avg_n(n INT)
LANGUAGE plpgsql AS $$
DECLARE
    s NUMERIC;
    a NUMERIC;
BEGIN
    s := (n * (n + 1)) / 2.0;
    a := s / n;
    RAISE NOTICE 'Sum: %, Avg: %', s, a;
END;
$$;

```

Slip 12

Section I: Data Structures-II

Q1) Write a program to implement various types of hash functions which are used to place the data in a hash table a. Division Method b. Mid Square Method c. Digit Folding Method. Accept n values from the user and display appropriate message in case of collision for each of the above functions.

```

#include <stdio.h>

#define SIZE 10

int divisionMethod(int key) { return key % SIZE; }

int midSquareMethod(int key) {
    int sq = key * key;
    int mid = (sq / 10) % 100;
    return mid % SIZE;
}

int digitFoldingMethod(int key) {
    int sum = 0;
    while (key > 0) {
        sum += key % 100;
        key /= 100;
    }
    return sum % SIZE;
}

```

```

}

void testHashing(int keys[], int n, int (*hashFunc)(int), const char* name) {
    int table[SIZE] = {0};
    printf("\n--- %s ---\n", name);
    for (int i = 0; i < n; i++) {
        int idx = hashFunc(keys[i]);
        if (table[idx] != 0)
            printf("Collision detected for Key %d at Index %d!\n", keys[i], idx);
        else table[idx] = keys[i];
    }
}

int main() {
    int keys[] = {12, 22, 42, 55};
    testHashing(keys, 4, divisionMethod, "Division Method");
    testHashing(keys, 4, midSquareMethod, "Mid Square Method");
    testHashing(keys, 4, digitFoldingMethod, "Digit Folding Method");
    return 0;
}

```

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an adjacency list. Implement function to traverse the graph using Depth First Search (DFS) traversal.

(Same code as Slip 4 OR Q1)

Section II: Database Management Systems II

Q2) Database setup:

```

CREATE TABLE PROJECT (
    pno INT PRIMARY KEY,
    p_name CHAR(30),
    ptype CHAR(20),
    duration INT
);

CREATE TABLE EMPLOYEE (
    eno INT PRIMARY KEY,
    e_name CHAR(20),
    qualification CHAR(15),
    joindate DATE
);

CREATE TABLE PROJECT_EMPLOYEE (
    pno INT REFERENCES PROJECT(pno),
    eno INT REFERENCES EMPLOYEE(en),
    start_date DATE,
    no_of_hours_worked INT,
    PRIMARY KEY (pno, eno)
);

```

1. Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message.

(Same code as Slip 1 DBMS Q2 Part 1 OR)

OR

1. Write a stored function to accept project name as input and print the names of employees working on the project. Raise an exception for an invalid project name.

```

CREATE OR REPLACE PROCEDURE print_proj_employees(p_pname CHAR(30))
LANGUAGE plpgsql AS $$
DECLARE
    r RECORD;
    v_found BOOLEAN := FALSE;
BEGIN
    FOR r IN
        SELECT e.e_name FROM EMPLOYEE e
        JOIN PROJECT_EMPLOYEE pe ON e.eno = pe.eno
        JOIN PROJECT p ON pe.pno = p.pno
        WHERE p.p_name = p_pname
    LOOP
        v_found := TRUE;
        RAISE NOTICE 'Employee Name: %', r.e_name;
    END LOOP;
END;

```

```

IF NOT v_found THEN
    RAISE EXCEPTION 'Invalid project name or no employees assigned.';
END IF;
END;
$$;

```

2. Write a procedure to accept value of m and n and display the total count of odd numbers from m to n.

```

CREATE OR REPLACE PROCEDURE count_odds_m_to_n(m INT, n INT)
LANGUAGE plpgsql AS $$
DECLARE
    cnt INT := 0;
    i INT;
BEGIN
    FOR i IN m..n LOOP
        IF i % 2 <> 0 THEN
            cnt := cnt + 1;
        END IF;
    END LOOP;
    RAISE NOTICE 'Total odd numbers between % and %: %', m, n, cnt;
END;
$$;

```

Slip 13

Section I: Data Structures-II

Q1) Write a C program to implement a hash table using separate chaining with linked lists with following operation. a. Insert a key b. Search a key c. Display the hash table

(Same code as Slip 9 Main Q1)

OR

Q1) Write a C program that accepts the vertices and edges of a graph. Create and display adjacency list also print indegree, outdegree and total degree of all vertex of graph.

(Same code as Slip 2 OR Q1)

Section II: Database Management Systems II

Q2) Database setup:

```

CREATE TABLE BUS (
    bus_no INT PRIMARY KEY,
    capacity INT NOT NULL,
    depot_name VARCHAR(20)
);

CREATE TABLE ROUTE (
    route_no INT PRIMARY KEY,
    source CHAR(20),
    destination CHAR(20),
    no_of_stations INT
);

CREATE TABLE DRIVER (
    driver_no INT PRIMARY KEY,
    driver_name CHAR(20),
    license_no INT UNIQUE,
    address CHAR(20),
    d_age INT,
    salary FLOAT
);

CREATE TABLE BUS_DRIVER (
    bus_no INT REFERENCES BUS(bus_no),
    driver_no INT REFERENCES DRIVER(driver_no),
    duty_date DATE,
    shift INT CHECK (shift IN (1, 2)),
    PRIMARY KEY(bus_no, driver_no, duty_date)
);

```

1. Define a trigger before insert the record of driver if the age is not between 18 and 50, raise an error message "invalid entry".

```

CREATE OR REPLACE FUNCTION check_driver_age()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.d_age < 18 OR NEW.d_age > 50 THEN

```

```

        RAISE EXCEPTION 'invalid entry';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_driver_age
BEFORE INSERT ON DRIVER
FOR EACH ROW
EXECUTE FUNCTION check_driver_age();

```

OR

1. Write a stored function to accept the bus_no and date and print its allotted drivers. Raise an exception in case of invalid bus number.

```

CREATE OR REPLACE PROCEDURE print_allotted_drivers(p_busno INT, p_date DATE)
LANGUAGE plpgsql AS $$
DECLARE
    r RECORD;
    v_found BOOLEAN := FALSE;
BEGIN
    FOR r IN
        SELECT d.driver_name FROM DRIVER d
        JOIN BUS_DRIVER bd ON d.driver_no = bd.driver_no
        WHERE bd.bus_no = p_busno AND bd.duty_date = p_date
    LOOP
        v_found := TRUE;
        RAISE NOTICE 'Driver: %', r.driver_name;
    END LOOP;

    IF NOT v_found THEN
        RAISE EXCEPTION 'Invalid bus number or no driver allotted on this date.';
    END IF;
END;
$$;

```

2. Write a procedure to accept value of n and display all odd numbers from 1 to n.

```

CREATE OR REPLACE PROCEDURE print_odds_1_to_n(n INT)
LANGUAGE plpgsql AS $$
DECLARE
    i INT;
BEGIN
    FOR i IN 1..n LOOP
        IF i % 2 <> 0 THEN
            RAISE NOTICE '%', i;
        END IF;
    END LOOP;
END;
$$;

```

Slip 14

Section I: Data Structures-II

Q1) Write a C program to maintain a phonebook using Binary Search Tree by name where each node contains contact name and phone number.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char name[30];
    char phone[15];
    struct Node *left, *right;
} Node;

Node* insert(Node* root, char name[], char phone[]) {
    if (!root) {
        Node* n = (Node*)malloc(sizeof(Node));
        strcpy(n->name, name);
        strcpy(n->phone, phone);
        n->left = n->right = NULL;
        return n;
    }
    if (strcmp(name, root->name) < 0)
        root->left = insert(root->left, name, phone);
    else if (strcmp(name, root->name) > 0)

```

```

        root->right = insert(root->right, name, phone);
    }
    return root;
}

void inorder(Node* root) {
    if (root) {
        inorder(root->left);
        printf("Name: %s | Phone: %s\n", root->name, root->phone);
        inorder(root->right);
    }
}

int main() {
    Node* root = NULL;
    root = insert(root, "Amit", "9876543210");
    insert(root, "Rahul", "8765432109");
    insert(root, "Priya", "7654321098");

    printf("Phonebook Contacts:\n");
    inorder(root);
    return 0;
}

```

OR

Q1) Write a C program to implement a hash table using open addressing with linear probing. perform the following operations. (Assume all keys are positive integers) a. Insert b. Search c. Display

```

#include <stdio.h>
#define SIZE 10

int hash[SIZE] = {0};

void insert(int key) {
    int idx = key % SIZE;
    int i = 0;
    while (hash[(idx + i) % SIZE] != 0 && i < SIZE) i++;
    if (i < SIZE) hash[(idx + i) % SIZE] = key;
    else printf("Table full!\n");
}

void search(int key) {
    int idx = key % SIZE;
    int i = 0;
    while (hash[(idx + i) % SIZE] != key && hash[(idx + i) % SIZE] != 0 && i < SIZE) i++;
    if (hash[(idx + i) % SIZE] == key) printf("Key %d Found!\n", key);
    else printf("Key %d Not Found!\n", key);
}

void display() {
    for (int i = 0; i < SIZE; i++) printf("Index %d : %d\n", i, hash[i]);
}

int main() {
    insert(12); insert(22); insert(32);
    display();
    search(22);
    return 0;
}

```

Section II: Database Management Systems-II

Q2) Database setup:

```

CREATE TABLE PROJECT (
    pno INT PRIMARY KEY,
    p_name CHAR(30),
    ptype CHAR(20),
    duration INT
);

CREATE TABLE EMPLOYEE (
    eno INT PRIMARY KEY,
    e_name CHAR(20),
    qualification CHAR(15),
    joindate DATE
);

```

1. Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message.

(Same code as Slip 1 DBMS Q2 Part 1 OR)

OR

1. Write a stored function to accept project name as input and print the names of employees working on the project. Raise an exception for an invalid project name.

(Same code as Slip 12 DBMS Q2 Part 1 OR)

2. Write a procedure to insert the values in project table.

```
CREATE OR REPLACE PROCEDURE insert_project(  
    p_pno INT, p_pname CHAR(30), p_ptype CHAR(20), p_duration INT  
)  
LANGUAGE plpgsql AS $$  
BEGIN  
    INSERT INTO PROJECT(pno, p_name, ptype, duration)  
    VALUES (p_pno, p_pname, p_ptype, p_duration);  
    RAISE NOTICE 'Project inserted successfully.';  
END;  
$$;
```

Slip 15

Section I: Data Structures-II

Q1) Write a C program to implement a hash table using Open Addressing with Quadratic Probing. Perform the following operations. (Assume all keys are positive integers) a. Insert b. Search c. Display

```
#include <stdio.h>  
#define SIZE 10  
  
int hash[SIZE] = {0};  
  
void insert(int key) {  
    int idx = key % SIZE;  
    int i = 0;  
    while (hash[(idx + i * i) % SIZE] != 0 && i < SIZE) i++;  
    if (i < SIZE) hash[(idx + i * i) % SIZE] = key;  
    else printf("Table full or Quadratic probing limit reached!\n");  
}  
  
void search(int key) {  
    int idx = key % SIZE;  
    int i = 0;  
    while (hash[(idx + i * i) % SIZE] != key && hash[(idx + i * i) % SIZE] != 0 && i < SIZE) i++;  
    if (hash[(idx + i * i) % SIZE] == key) printf("Key %d Found!\n", key);  
    else printf("Key %d Not Found!\n", key);  
}  
  
void display() {  
    for (int i = 0; i < SIZE; i++) printf("Index %d : %d\n", i, hash[i]);  
}  
  
int main() {  
    insert(12); insert(22); insert(32);  
    display();  
    search(22);  
    return 0;  
}
```

OR

Q1) Implement a Binary search tree (BST) library (btree.h) with operations - create, search, insert, inorder, preorder and postorder. Write a menu driven program that performs the above operations.

(Same code as Slip 1 Main Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Area (  
    aname VARCHAR(20) PRIMARY KEY,  
    area_type VARCHAR(5) CHECK (area_type IN ('urban', 'rural'))  
);  
  
CREATE TABLE Person (  
    pno INT PRIMARY KEY,  
    p_name VARCHAR(30),  
    p_ptype VARCHAR(20),  
    p_duration INT
```

```

pnumber INT PRIMARY KEY,
pname VARCHAR(20),
birthdate DATE,
income NUMERIC,
aname VARCHAR(20) REFERENCES Area(aname)
);

```

1. Write a cursor to update the income of all people living in 'Urban' area by 10%.

```

CREATE OR REPLACE PROCEDURE update_urban_income()
LANGUAGE plpgsql AS $$
DECLARE
    v_pnumber INT;
    cur CURSOR FOR
        SELECT p.pnumber FROM Person p
        JOIN Area a ON p.aname = a.aname
        WHERE LOWER(a.area_type) = 'urban';
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_pnumber;
        EXIT WHEN NOT FOUND;
        UPDATE Person SET income = income * 1.10 WHERE pnumber = v_pnumber;
    END LOOP;
    CLOSE cur;
    RAISE NOTICE 'Urban residents income updated by 10%.';
END;
$$;

```

OR

1. Write a trigger before insert the record of person if the person number is negative raise the message "Invalid Number".

```

CREATE OR REPLACE FUNCTION check_person_num()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.pnumber < 0 THEN
        RAISE EXCEPTION 'Invalid Number';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_person_num
BEFORE INSERT ON Person
FOR EACH ROW
EXECUTE FUNCTION check_person_num();

```

2. Write a procedure to display all odd numbers from 1 to 100.

```

CREATE OR REPLACE PROCEDURE print_odds_1_100()
LANGUAGE plpgsql AS $$
DECLARE
    i INT;
BEGIN
    FOR i IN 1..100 LOOP
        IF i % 2 <> 0 THEN
            RAISE NOTICE '%', i;
        END IF;
    END LOOP;
END;
$$;

```

Slip 16

Section I: Data Structures-II

Q1) Write a program to sort n randomly generated elements using Heapsort method.

(Same code as Slip 8 Main Q1)

OR

Q1) Write a C program for the implementation of Dijkstra's shortest path algorithm for finding shortest path from a given source vertex using adjacency cost matrix.

(Same code as Slip 7 OR Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Student (  
    rollno INT PRIMARY KEY,  
    name VARCHAR(30),  
    address VARCHAR(50),  
    class VARCHAR(10)  
);  
  
CREATE TABLE Subject (  
    scode VARCHAR(10) PRIMARY KEY,  
    subject_name VARCHAR(20)  
);  
  
CREATE TABLE Student_Subject (  
    rollno INT REFERENCES Student(rollno),  
    scode VARCHAR(10) REFERENCES Subject(scode),  
    marks_scored INT,  
    PRIMARY KEY(rollno, scode)  
);
```

1) Define a trigger before deleting a student record from the student table. Raise a notice and display the message "student record is being deleted".

(Same code as Slip 11 OR Q1)

OR

1) Write a stored function using cursors, to calculate total marks of each student and display it.

(Same code as Slip 11 Main Q1)

2) Write a procedure to accept a number and check the number is Positive and Negative or Zero.

(Same code as Slip 4 Q2 Part 2)

Slip 17

Section I: Data Structures-II

Q1) Write a C program that accepts the vertices and edges of a graph. Create and display adjacency list also print indegree, outdegree and total degree of all vertex of graph.

(Same code as Slip 2 OR Q1)

OR

Q1) Write a C program to find the minimum and maximum values in a Binary Search Tree.

(Same code as Slip 2 Main Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE STUDENT (  
    sreg_no INT PRIMARY KEY,  
    name CHAR(30),  
    class CHAR(10)  
);  
  
CREATE TABLE COMPETITION (  
    c_no INT PRIMARY KEY,  
    name CHAR(20),  
    C_type CHAR(15)  
);
```

1) Define a trigger before updating a competition table. Raise a notice and display message "competition record is being updated".

(Same code as Slip 5 OR Q1)

OR

2) Write a function using cursor which accepts a name of student and returns the total no of prizes won by that student in the year 2020.

(Same code as Slip 9 OR Q1)

2) Write a procedure to find sum of first 50 numbers.

```
CREATE OR REPLACE PROCEDURE sum_50_numbers()
LANGUAGE plpgsql AS $$
DECLARE
    s INT;
BEGIN
    s := (50 * 51) / 2;
    RAISE NOTICE 'Sum of first 50 numbers is %', s;
END;
$$;
```

Slip 18

Section I: Data Structures-II

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

(Same code as Slip 3 OR Q1)

OR

Q1) Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

(Same code as Slip 1 OR Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Item (
    itemno INT PRIMARY KEY,
    itemname VARCHAR(20)
);

CREATE TABLE Supplier (
    supplier_no INT PRIMARY KEY,
    supplier_name VARCHAR(20),
    city VARCHAR(20)
);

CREATE TABLE Item_Supplier (
    itemno INT REFERENCES Item(itemno),
    supplier_no INT REFERENCES Supplier(supplier_no),
    rate NUMERIC,
    quantity INT,
    PRIMARY KEY(itemno, supplier_no)
);
```

1. Write a cursor to display the names of items whose rate is more than 500.

```
CREATE OR REPLACE PROCEDURE items_rate_above_500()
LANGUAGE plpgsql AS $$
DECLARE
    v_iname VARCHAR(20);
    cur CURSOR FOR
        SELECT DISTINCT i.itemname FROM Item i
        JOIN Item_Supplier its ON i.itemno = its.itemno
        WHERE its.rate > 500;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_iname;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Item Name: %', v_iname;
    END LOOP;
    CLOSE cur;
END;
$$;
```

OR

1. Write a trigger before insert or update on rate field. If the rate is less than 50 then raise the appropriate exception.

```
CREATE OR REPLACE FUNCTION check_item_rate()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.rate < 50 THEN
        RAISE EXCEPTION 'Rate must be at least 50.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_rate
BEFORE INSERT OR UPDATE ON Item_Supplier
FOR EACH ROW
EXECUTE FUNCTION check_item_rate();
```

2. Write a procedure to accept three numbers and find the maximum number from it.

```
CREATE OR REPLACE PROCEDURE max_three_numbers(a INT, b INT, c INT)
LANGUAGE plpgsql AS $$
BEGIN
    RAISE NOTICE 'Maximum is %', GREATEST(a, b, c);
END;
$$;
```

Slip 19

Section I: Data Structures-II

Q1) Write a C program to maintain a phonebook using Binary Search Tree by name where each node contains contact name and phone number.

(Same code as Slip 14 Main Q1)

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an adjacency list. Implement function to traverse the graph using Breadth First Search (BFS) traversal.

(Same code as Slip 11 OR Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Project (
    pno INT PRIMARY KEY,
    pname CHAR(30) NOT NULL,
    ptype CHAR(20),
    duration INT CHECK (duration > 0)
);

CREATE TABLE Employee (
    eno INT PRIMARY KEY,
    ename CHAR(20),
    qualification CHAR(15),
    joining_date DATE
);
```

1) Write a stored function to accept eno as input parameter and count number of projects on which that employee is working.

(Same code as Slip 1 DBMS Q2 Part 1)

OR

1) Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message

(Same code as Slip 1 DBMS Q2 Part 1 OR)

2) Write a procedure to accept two numbers and find minimum and maximum from two numbers.

(Same code as Slip 3 Q2 Part 2)

Slip 20

Section I: Data Structures-II

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an adjacency list. Implement function to traverse the graph using Depth First Search (DFS) traversal.

(Same code as Slip 4 OR Q1)

OR

Q1) Write a C program for the Implementation of Kruskal's Minimum spanning tree algorithm.

(Same code as Slip 6 Main Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Department (  
    dno INT PRIMARY KEY,  
    dname VARCHAR(20),  
    city VARCHAR(20)  
);  
  
CREATE TABLE Employee (  
    eno INT PRIMARY KEY,  
    ename VARCHAR(20),  
    salary NUMERIC,  
    dno INT REFERENCES Department(dno)  
);
```

1) Write a function to accept department name and display the maximum salary of an employee in that department.

```
CREATE OR REPLACE FUNCTION get_max_sal(p_dname VARCHAR)  
RETURNS NUMERIC AS $$  
DECLARE  
    max_sal NUMERIC;  
BEGIN  
    SELECT MAX(e.salary) INTO max_sal  
    FROM Employee e JOIN Department d ON e.dno = d.dno  
    WHERE d.dname = p_dname;  
    RETURN COALESCE(max_sal, 0);  
END;  
$$ LANGUAGE plpgsql;
```

OR

1) Write a trigger before insert/update on an employee record. Raise exception if salary <0.

```
CREATE OR REPLACE FUNCTION check_emp_salary()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.salary < 0 THEN  
        RAISE EXCEPTION 'Salary cannot be negative.';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_emp_salary  
BEFORE INSERT OR UPDATE ON Employee  
FOR EACH ROW  
EXECUTE FUNCTION check_emp_salary();
```

2) Write a procedure to accept a number and display multiplication table of a number.

```
CREATE OR REPLACE PROCEDURE print_table(num INT)  
LANGUAGE plpgsql AS $$  
DECLARE  
    i INT;  
BEGIN  
    FOR i IN 1..10 LOOP  
        RAISE NOTICE '% x % = %', num, i, (num * i);  
    END LOOP;  
END;  
$$;
```

Section I: Data Structures-II

Q1) Write a C program for the implementation of Topological sorting.

(Same code as Slip 7 Main Q1)

OR

Q1) Write a C program to implement a hash table using open addressing with linear probing. perform the following operations. (Assume all keys are positive integers) a. Insert b. Search c. Display

(Same code as Slip 14 OR Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Hospital (  
    h_no INT PRIMARY KEY,  
    h_name VARCHAR(20),  
    city VARCHAR(10)  
);  
  
CREATE TABLE Doctor (  
    d_no INT PRIMARY KEY,  
    d_name VARCHAR(30),  
    specialization VARCHAR(35),  
    phone_no VARCHAR(15),  
    charges INT,  
    h_no INT REFERENCES Hospital(h_no)  
);
```

1) Write a function which will accept the name of the hospital and calculate the average charges of doctors visiting that hospital.

```
CREATE OR REPLACE FUNCTION avg_doc_charges(p_hname VARCHAR)  
RETURNS NUMERIC AS $$  
DECLARE  
    avg_chg NUMERIC;  
BEGIN  
    SELECT AVG(d.charges) INTO avg_chg  
    FROM Doctor d JOIN Hospital h ON d.h_no = h.h_no  
    WHERE h.h_name = p_hname;  
    RETURN COALESCE(avg_chg, 0);  
END;  
$$ LANGUAGE plpgsql;
```

OR

1) Write a trigger before insert/update on Doctor. Raise exception if charges are <0.

```
CREATE OR REPLACE FUNCTION check_doc_charges()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.charges < 0 THEN  
        RAISE EXCEPTION 'Doctor charges cannot be negative.';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_doc_charges  
BEFORE INSERT OR UPDATE ON Doctor  
FOR EACH ROW  
EXECUTE FUNCTION check_doc_charges();
```

2) Write a procedure to insert the values in Doctor table.

```
CREATE OR REPLACE PROCEDURE insert_doctor(  
    p_dno INT, p_dname VARCHAR(30), p_spec VARCHAR(35), p_phone VARCHAR(15), p_charges INT, p_hno INT  
)  
LANGUAGE plpgsql AS $$  
BEGIN  
    INSERT INTO Doctor(d_no, d_name, specialization, phone_no, charges, h_no)  
    VALUES (p_dno, p_dname, p_spec, p_phone, p_charges, p_hno);  
    RAISE NOTICE 'Doctor record inserted successfully.';  
END;  
$$;
```

Slip 22

Section I: Data Structures-II

Q1) Write a C program to implement a hash table using Open Addressing with Quadratic Probing. Perform the following operations. (Assume all keys are positive integers) a. Insert b. Search c. Display

(Same code as Slip 15 Main Q1)

OR

Q1) Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

(Same code as Slip 1 OR Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Project (  
    pno INT PRIMARY KEY,  
    pname CHAR(30) NOT NULL,  
    ptype CHAR(20),  
    duration INT CHECK (duration > 0)  
);
```

```
CREATE TABLE Employee (  
    eno INT PRIMARY KEY,  
    ename CHAR(20),  
    qualification CHAR(15),  
    joining_date DATE  
);
```

1) Write a stored function to accept eno as input parameter and count number of projects on which that employee is working.

(Same code as Slip 1 DBMS Q2 Part 1)

OR

1) Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message

(Same code as Slip 1 DBMS Q2 Part 1 OR)

2) Write a procedure to display first 20 odd numbers.

```
CREATE OR REPLACE PROCEDURE print_first_20_odds()  
LANGUAGE plpgsql AS $$  
DECLARE  
    count INT := 0;  
    num INT := 1;  
BEGIN  
    WHILE count < 20 LOOP  
        RAISE NOTICE '%', num;  
        num := num + 2;  
        count := count + 1;  
    END LOOP;  
END;  
$$;
```

Slip 23

Section I: Data Structures-II

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

(Same code as Slip 3 OR Q1)

OR

Q1) Write a C program for the implementation of Topological sorting.

(Same code as Slip 7 Main Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Car (  
    c_no INT PRIMARY KEY,  
    owner VARCHAR(20),  
    model VARCHAR(10),  
    color VARCHAR(10)  
);  
  
CREATE TABLE Driver (  
    driver_no INT PRIMARY KEY,  
    driver_name VARCHAR(20),  
    license_no INT,  
    d_age INT,  
    salary FLOAT  
);  
  
CREATE TABLE Car_Driver (  
    c_no INT REFERENCES Car(c_no),  
    driver_no INT REFERENCES Driver(driver_no),  
    PRIMARY KEY(c_no, driver_no)  
);
```

1) Write a cursor which accepts the driver name and prints the details of all cars that this driver has driven, if the driver name is invalid, print an appropriate message.

```
CREATE OR REPLACE PROCEDURE get_cars_by_driver(p_dname VARCHAR)  
LANGUAGE plpgsql AS $$  
DECLARE  
    r RECORD;  
    v_found BOOLEAN := FALSE;  
    cur CURSOR FOR  
        SELECT c.* FROM Car c  
        JOIN Car_Driver cd ON c.c_no = cd.c_no  
        JOIN Driver d ON cd.driver_no = d.driver_no  
        WHERE d.driver_name = p_dname;  
BEGIN  
    OPEN cur;  
    LOOP  
        FETCH cur INTO r;  
        EXIT WHEN NOT FOUND;  
        v_found := TRUE;  
        RAISE NOTICE 'Car No: %, Model: %, Color: %', r.c_no, r.model, r.color;  
    END LOOP;  
    CLOSE cur;  
  
    IF NOT v_found THEN  
        RAISE NOTICE 'Invalid driver name or driver has driven no cars.';  
    END IF;  
END;  
$$;
```

OR

1) Write a trigger before insert/update on Driver. Raise exception if driver age is < 21.

```
CREATE OR REPLACE FUNCTION check_driver_min_age()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.d_age < 21 THEN  
        RAISE EXCEPTION 'Driver age must be at least 21.';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_driver_min_age  
BEFORE INSERT OR UPDATE ON Driver  
FOR EACH ROW  
EXECUTE FUNCTION check_driver_min_age();
```

2) Write a procedure to find sum of first 100 numbers.

```
CREATE OR REPLACE PROCEDURE sum_100_numbers()  
LANGUAGE plpgsql AS $$  
DECLARE  
    s INT;  
BEGIN  
    s := (100 * 101) / 2;
```

```
RAISE NOTICE 'Sum of first 100 numbers is %', s;
END;
$$;
```

Slip 24

Section I: Data Structures-II

Q1) Write a C program for the implementation of Floyd Warshall's algorithm for finding all pairs shortest path using adjacency cost matrix.

(Same code as Slip 10 Main Q1)

OR

Q1) Write a C program to maintain a phonebook using Binary Search Tree by name where each node contains contact name and phone number.

(Same code as Slip 14 Main Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Department (
    dno INT PRIMARY KEY,
    dname VARCHAR(20),
    city VARCHAR(20)
);

CREATE TABLE Employee (
    eno INT PRIMARY KEY,
    ename VARCHAR(20),
    salary NUMERIC,
    dno INT REFERENCES Department(dno)
);
```

1) Write a function to accept department name as input and display employee name along with salary of that department.

```
CREATE OR REPLACE PROCEDURE display_dept_employees(p_dname VARCHAR)
LANGUAGE plpgsql AS $$
DECLARE
    r RECORD;
BEGIN
    FOR r IN
        SELECT e.ename, e.salary FROM Employee e
        JOIN Department d ON e.dno = d.dno
        WHERE d.dname = p_dname
    LOOP
        RAISE NOTICE 'Employee: %, Salary: %', r.ename, r.salary;
    END LOOP;
END;
$$;
```

OR

1) Write a trigger after insert on an employee record. Display appropriate message when the record is inserted.

```
CREATE OR REPLACE FUNCTION notify_emp_inserted()
RETURNS TRIGGER AS $$
BEGIN
    RAISE NOTICE 'New employee record inserted successfully!';
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_after_emp_insert
AFTER INSERT ON Employee
FOR EACH ROW
EXECUTE FUNCTION notify_emp_inserted();
```

2) Write a stored procedure to accept value of n and display first n even numbers.

```
CREATE OR REPLACE PROCEDURE print_n_evens(n INT)
LANGUAGE plpgsql AS $$
DECLARE
    count INT := 0;
    num INT := 2;
```

```

BEGIN
    WHILE count < n LOOP
        RAISE NOTICE '%', num;
        num := num + 2;
        count := count + 1;
    END LOOP;
END;
$$;

```

Slip 25

Section I: Data Structures-II

Q1) Write a C program for the implementation of Dijkstra's shortest path algorithm for finding shortest path from a given source vertex using adjacency cost matrix.

(Same code as Slip 7 OR Q1)

OR

Q1) Implement a Binary search tree (BST) library (btree.h) with operations - create, search, insert, inorder, preorder and postorder. Write a menu driven program that performs the above operations.

(Same code as Slip 1 Main Q1)

Section II: Database Management Systems-II

Q2) Database setup:

```

CREATE TABLE Customer (
    cno INT PRIMARY KEY,
    cname VARCHAR(20),
    city VARCHAR(20)
);

CREATE TABLE Account (
    a_no INT PRIMARY KEY,
    a_type VARCHAR(10),
    opening_date DATE,
    balance NUMERIC,
    cno INT REFERENCES Customer(cno)
);

```

1. Write a function using cursor which accepts city name as input and prints the details of all customers in that city.

```

CREATE OR REPLACE PROCEDURE get_customers_by_city(p_city VARCHAR)
LANGUAGE plpgsql AS $$
DECLARE
    r RECORD;
    cur CURSOR FOR SELECT * FROM Customer WHERE city = p_city;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO r;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'ID: %, Name: %, City: %', r.cno, r.cname, r.city;
    END LOOP;
    CLOSE cur;
END;
$$;

```

OR

1. Write a trigger which does not allow deletion of accounts of specific type.

```

CREATE OR REPLACE FUNCTION prevent_acc_deletion()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.a_type = 'Savings' THEN
        RAISE EXCEPTION 'Deletion of Savings account is not allowed!';
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_del_acc
BEFORE DELETE ON Account

```

```
FOR EACH ROW  
EXECUTE FUNCTION prevent_acc_deletion();
```

2. Write a procedure to display all customers from 'Pune' city.

```
CREATE OR REPLACE PROCEDURE pune_customers()  
LANGUAGE plpgsql AS $$  
DECLARE  
    r RECORD;  
BEGIN  
    FOR r IN SELECT * FROM Customer WHERE city = 'Pune' LOOP  
        RAISE NOTICE 'Customer ID: %, Name: %', r.cno, r.cname;  
    END LOOP;  
END;  
$$;
```

Slip 26

Section I: Data Structures-II

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an adjacency matrix. Implement function to traverse the graph using Depth First Search (DFS) traversal.

```
#include <stdio.h>  
  
int adj[20][20], visited[20], v;  
  
void dfs(int start) {  
    visited[start] = 1;  
    printf("%d ", start);  
    for (int i = 0; i < v; i++) {  
        if (adj[start][i] == 1 && !visited[i])  
            dfs(i);  
    }  
}  
  
int main() {  
    int e, src, dest;  
    printf("Enter vertices and edges: ");  
    scanf("%d %d", &v, &e);  
    for (int i = 0; i < e; i++) {  
        printf("Enter edge (src dest): ");  
        scanf("%d %d", &src, &dest);  
        adj[src][dest] = adj[dest][src] = 1;  
    }  
    printf("DFS Traversal: ");  
    dfs(0);  
    printf("\n");  
    return 0;  
}
```

OR

Q1) Write a C program to find the minimum and maximum values in a Binary Search Tree.

(Same code as Slip 2 Main Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Item (  
    itemno INT PRIMARY KEY,  
    Itemname VARCHAR(20),  
    quantity INT  
);  
  
CREATE TABLE Supplier (  
    suppliernumber INT PRIMARY KEY,  
    Supplier_name VARCHAR(20),  
    city VARCHAR(20)  
);  
  
CREATE TABLE Item_Supplier (  
    itemno INT REFERENCES Item(itemno),  
    suppliernumber INT REFERENCES Supplier(suppliernumber),  
    rate NUMERIC,  
    PRIMARY KEY(itemno, suppliernumber)  
);
```

1) Write a stored function using cursors, to accept Item name from the user and display the Rate and Supplier Name for that Item.

```
CREATE OR REPLACE PROCEDURE get_item_supplier_details(p_iname VARCHAR)
LANGUAGE plpgsql AS $$
DECLARE
    v_rate NUMERIC;
    v_sname VARCHAR(20);
    cur CURSOR FOR
        SELECT its.rate, s.Supplier_name
        FROM Item i
        JOIN Item_Supplier its ON i.itemno = its.itemno
        JOIN Supplier s ON its.supplierno = s.supplierno
        WHERE i.Itemname = p_iname;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_rate, v_sname;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Supplier: %, Rate: %', v_sname, v_rate;
    END LOOP;
    CLOSE cur;
END;
$$;
```

OR

2) Write a trigger before update on rate field. If the difference in the old rate and new rate is more than Rs 2000, raise an exception and display the corresponding message.

```
CREATE OR REPLACE FUNCTION check_rate_diff()
RETURNS TRIGGER AS $$
BEGIN
    IF ABS(NEW.rate - OLD.rate) > 2000 THEN
        RAISE EXCEPTION 'Rate difference cannot exceed Rs 2000!';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_rate_diff
BEFORE UPDATE ON Item_Supplier
FOR EACH ROW
EXECUTE FUNCTION check_rate_diff();
```

3) Write a procedure to accept and display addition, subtraction, multiplication and division of two numbers.

```
CREATE OR REPLACE PROCEDURE calc_four_ops(a NUMERIC, b NUMERIC)
LANGUAGE plpgsql AS $$
BEGIN
    RAISE NOTICE 'Add: %', (a + b);
    RAISE NOTICE 'Sub: %', (a - b);
    RAISE NOTICE 'Mul: %', (a * b);
    IF b <> 0 THEN
        RAISE NOTICE 'Div: %', (a / b);
    ELSE
        RAISE NOTICE 'Div by Zero Error';
    END IF;
END;
$$;
```

Slip 27

Section I: Data Structures-II

Q1) Write a C program for the Implementation of Prim's Minimum spanning tree algorithm.

(Same code as Slip 5 OR Q1)

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an adjacency matrix. Implement function to traverse the graph using Breadth First Search (BFS) traversal.

(Same code as Slip 3 Main Q1)

Section II: Database Management Systems II

Q2) Database setup:

```

CREATE TABLE Student (
    rollno INT PRIMARY KEY,
    name VARCHAR(30),
    class VARCHAR(10)
);

CREATE TABLE Subject (
    Scode VARCHAR(10) PRIMARY KEY,
    subject_name VARCHAR(20)
);

CREATE TABLE Student_Subject (
    rollno INT REFERENCES Student(rollno),
    Scode VARCHAR(10) REFERENCES Subject(Scode),
    marks_scored INT,
    PRIMARY KEY(rollno, Scode)
);

```

1. Write a stored function using cursors, to accept class from the user and display the details of the students of that class.

```

CREATE OR REPLACE PROCEDURE get_students_by_class(p_class VARCHAR)
LANGUAGE plpgsql AS $$
DECLARE
    r RECORD;
    cur CURSOR FOR SELECT * FROM Student WHERE class = p_class;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO r;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Roll No: %, Name: %, Class: %', r.rollno, r.name, r.class;
    END LOOP;
    CLOSE cur;
END;
$$;

```

OR

1. Write a trigger before insert/update the marks_scored. Raise exception if Marks are negative.

```

CREATE OR REPLACE FUNCTION check_marks()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.marks_scored < 0 THEN
        RAISE EXCEPTION 'Marks cannot be negative!';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_marks
BEFORE INSERT OR UPDATE ON Student_Subject
FOR EACH ROW
EXECUTE FUNCTION check_marks();

```

2. Write a procedure to accept and display addition, subtraction and division of two numbers. Handle division by zero error for division operation. (Use raise).

```

CREATE OR REPLACE PROCEDURE calc_three_ops(a NUMERIC, b NUMERIC)
LANGUAGE plpgsql AS $$
BEGIN
    RAISE NOTICE 'Addition: %', (a + b);
    RAISE NOTICE 'Subtraction: %', (a - b);
    IF b = 0 THEN
        RAISE EXCEPTION 'Division by Zero Error!';
    ELSE
        RAISE NOTICE 'Division: %', (a / b);
    END IF;
END;
$$;

```

Slip 28

Section I: Data Structures-II

Q1) Write a C program to implement a hash table using Open Addressing with Quadratic Probing. Perform the following operations. (Assume all keys are positive integers) a. Insert b. Delete c. Display

```

#include <stdio.h>
#define SIZE 10

```

```

int hash[SIZE] = {0};

void insert(int key) {
    int idx = key % SIZE, i = 0;
    while (hash[(idx + i * i) % SIZE] != 0 && hash[(idx + i * i) % SIZE] != -1 && i < SIZE) i++;
    if (i < SIZE) hash[(idx + i * i) % SIZE] = key;
}

void deleteKey(int key) {
    int idx = key % SIZE, i = 0;
    while (hash[(idx + i * i) % SIZE] != key && hash[(idx + i * i) % SIZE] != 0 && i < SIZE) i++;
    if (hash[(idx + i * i) % SIZE] == key) {
        hash[(idx + i * i) % SIZE] = -1; // -1 represents deleted slot
        printf("Key %d Deleted!\n", key);
    } else printf("Key %d Not Found!\n", key);
}

void display() {
    for (int i = 0; i < SIZE; i++) printf("Index %d : %d\n", i, hash[i]);
}

int main() {
    insert(12); insert(22);
    display();
    deleteKey(22);
    display();
    return 0;
}

```

OR

Q1) Write a C program for the Implementation of Prim's Minimum spanning tree algorithm.

(Same code as Slip 5 OR Q1)

Section II: Database Management Systems-II

Q2) Database setup:

```

CREATE TABLE Company (
    Name VARCHAR(30) PRIMARY KEY,
    address VARCHAR(50),
    city VARCHAR(20),
    share_value NUMERIC
);

CREATE TABLE Person (
    pname VARCHAR(30) PRIMARY KEY,
    pcity VARCHAR(20)
);

CREATE TABLE Company_Person (
    Name VARCHAR(30) REFERENCES Company(Name),
    pname VARCHAR(30) REFERENCES Person(pname),
    No_of_shares INT,
    PRIMARY KEY(Name, pname)
);

```

1) Write a stored function to update the share_values by 20% for Person

```

CREATE OR REPLACE FUNCTION update_share_val()
RETURNS VOID AS $$
BEGIN
    UPDATE Company SET share_value = share_value * 1.20;
    RAISE NOTICE 'Share values updated by 20%';
END;
$$ LANGUAGE plpgsql;

```

OR

1) Write a trigger before deleting company record. Display appropriate message to the user.

```

CREATE OR REPLACE FUNCTION notify_company_del()
RETURNS TRIGGER AS $$
BEGIN
    RAISE NOTICE 'Company record is being deleted from system.';
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

```

```
CREATE TRIGGER trg_del_company
BEFORE DELETE ON Company
FOR EACH ROW
EXECUTE FUNCTION notify_company_del();
```

2) Write a procedure to insert the values in person table.

```
CREATE OR REPLACE PROCEDURE insert_person(p_pname VARCHAR(30), p_pcity VARCHAR(20))
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO Person(pname, pcity) VALUES (p_pname, p_pcity);
    RAISE NOTICE 'Person record inserted successfully.';
END;
$$;
```

Slip 29

Section I: Data Structures-II

Q1) Write a program to sort n randomly generated elements using Heapsort method.

(Same code as Slip 8 Main Q1)

OR

Q1) Write a C program for the Implementation of Kruskal's Minimum spanning tree algorithm.

(Same code as Slip 6 Main Q1)

Section II: Database Management Systems II

Q2) Database setup:

```
CREATE TABLE Area (
    aid INT PRIMARY KEY,
    aname VARCHAR(20),
    area_type VARCHAR(5) CHECK (area_type IN ('urban', 'rural'))
);

CREATE TABLE Person (
    pno INT PRIMARY KEY,
    pname VARCHAR(20),
    birthdate DATE,
    income NUMERIC,
    aid INT REFERENCES Area(aid)
);
```

1) Write a cursor to display the names of persons living in urban area.

```
CREATE OR REPLACE PROCEDURE get_urban_persons()
LANGUAGE plpgsql AS $$
DECLARE
    v_pname VARCHAR(20);
    cur CURSOR FOR
        SELECT p.pname FROM Person p
        JOIN Area a ON p.aid = a.aid
        WHERE LOWER(a.area_type) = 'urban';
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_pname;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Person Name: %', v_pname;
    END LOOP;
    CLOSE cur;
END;
$$;
```

OR

1) Write a trigger before deleting a person's record from the person's table. Raise a notice and display the message "person record is being deleted".

```
CREATE OR REPLACE FUNCTION notify_person_del()
RETURNS TRIGGER AS $$
BEGIN
    RAISE NOTICE 'person record is being deleted';
```

```

RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_del_person
BEFORE DELETE ON Person
FOR EACH ROW
EXECUTE FUNCTION notify_person_del();

```

2) Write a procedure to accept three numbers from user and display maximum and minimum of three numbers.

(Same code as Slip 5 Q2 Part 2)

Slip 30

Section I: Data Structures-II

Q1) Write a C program for the Implementation of Kruskal's Minimum spanning tree algorithm.

(Same code as Slip 6 Main Q1)

OR

Q1) Write a C program to implement a hash table using open addressing with linear probing. perform the following operations. (Assume all keys are positive integers) a.Insert b.Search c.Display

(Same code as Slip 14 OR Q1)

Section II: Database Management Systems II

Q2) Database setup:

```

CREATE TABLE Student (
    Roll_No INT PRIMARY KEY,
    Sname VARCHAR(20),
    Sclass CHAR(10)
);

CREATE TABLE Teacher (
    T_No INT PRIMARY KEY,
    Tname CHAR(20),
    Experience INT
);

CREATE TABLE Student_Teacher (
    Roll_No INT REFERENCES Student(Roll_No),
    T_No INT REFERENCES Teacher(T_No),
    Subject VARCHAR(20),
    PRIMARY KEY(Roll_No, T_No, Subject)
);

```

1) Write a stored function to count the number of teachers having experience > 10 years

```

CREATE OR REPLACE FUNCTION count_exp_teachers()
RETURNS INT AS $$
DECLARE
    cnt INT;
BEGIN
    SELECT COUNT(*) INTO cnt FROM Teacher WHERE Experience > 10;
    RETURN cnt;
END;
$$ LANGUAGE plpgsql;

```

OR

1) Write a trigger before insert the record of the student in the Student table. If the Roll_No is less than or equal to zero then the trigger gets fired and displays the message "Invalid Roll Number".

```

CREATE OR REPLACE FUNCTION check_roll_no()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.Roll_No <= 0 THEN
        RAISE EXCEPTION 'Invalid Roll Number';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```
CREATE TRIGGER trg_check_roll  
BEFORE INSERT ON Student  
FOR EACH ROW  
EXECUTE FUNCTION check_roll_no();
```

2) Write a procedure to accept value of n from user and find sum and average of first n numbers.

(Same code as Slip 11 Q2 Part 2)